Evaluating the Effect of LoRA Rank on Preference Alignment in Small Language Models

Tanisha Ravindran
Iowa State University

Abstract

Low-Rank Adaptation (LoRA) allows language models to be fine-tuned without updating all their parameters, making it useful when computational resources are limited. In this project, I investigated how LoRA rank affects preference alignment when fine-tuning Qwen2.5-0.5B-Instruct using Direct Preference Optimization (DPO). I compared ranks 8 and 16 using 2,000 preference pairs from Anthropic HH-RLHF, repeating each configuration across three random seeds. Performance was evaluated on 200 held-out prompts per run using an automated reward model. Rank 16 achieved an average win rate of 48.83%, compared with 39.83% for rank 8. Although the results favored rank 16, they also highlighted limitations in the evaluation process and the importance of interpreting preference scores carefully.

1. Introduction

Fine-tuning language models can be computationally expensive, especially with limited GPU resources. LoRA reduces this cost by updating a smaller set of trainable parameters rather than the entire model. Its adapter rank controls the capacity of these updates. A higher rank allows more flexible adaptation but requires additional parameters.

Direct Preference Optimization (DPO) fine-tunes language models using preferred and rejected responses. Rather than training a separate reward model during optimization, DPO learns directly from these preference pairs.

This project examines whether increasing LoRA rank from 8 to 16 improves preference alignment for a small language model. My hypothesis was that rank 16 would perform better because it has greater capacity to learn preference patterns.

2. Experimental Methodology

2.1 Model and Training Setup

I used Qwen2.5-0.5B-Instruct with 2,000 single-turn preference pairs from the Anthropic HH-RLHF dataset.

The original baseline used LoRA rank 16, while rank 8 served as the comparison. The main settings were held constant: DPO beta of 0.1, learning rate of 5×10^{-5}, one epoch, and an effective batch size of 16.

LoRA scaling was set to twice the rank, with dropout fixed at 0.05. All experiments ran in Google Colab using an NVIDIA T4 GPU, with random seeds 0, 1, and 2 for each rank.

Table 1: Preference win rates across three seeds.

Seed	Rank 8	Rank 16
0	41.0%	47.0%
1	35.0%	45.0%
2	43.5%	54.5%
Mean	**39.83%**	**48.83%**
Sample SD	4.37 pp	5.01 pp

2.2 Evaluation Procedure

For each run, I generated responses from the fine-tuned model and the original base model on 200 evaluation prompts.

The responses were scored using the OpenAssistant DeBERTa-v3 reward model. A win was counted when the fine-tuned response received a higher score than the base model response for the same prompt.

$$\text{Win Rate} = \frac{\text{DPO Wins}}{200} \times 100 \tag{1}$$

I calculated the mean and sample standard deviation across three seeds for each rank.

2.3 Baseline Reproduction and Debugging

While reviewing the baseline code, I noticed that evaluation prompts overlapped with training examples. The script selected the last 200 examples from the 2,000-example training subset for evaluation.

I corrected this by selecting the next 200 examples instead. These prompts were held out from fine-tuning, although they still came from the dataset's training split.

The first full training attempt also failed with a CUDA out-of-memory error. I reduced the per-device batch size from 4 to 1 and increased gradient accumulation from 4 to 16, preserving the effective batch size while reducing memory usage.

The original project reported a baseline win rate of approximately 71%. Because I changed the evaluation procedure, that result is not directly comparable with the results reported here.

3. Results

Rank 16 performed better than rank 8 in all three runs, with an average improvement of 9 percentage points. The difference between configurations ranged from 6 to 11

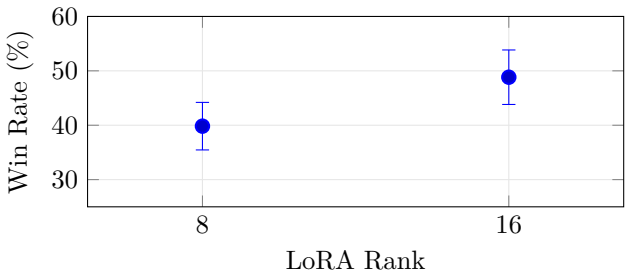


Figure 1: Mean win rate with ±1 sample standard deviation across three seeds.

percentage points across seeds.

However, neither configuration achieved an average win rate above 50%. Under the reward model used for evaluation, the fine-tuned responses received higher scores than the base model responses on fewer than half of the prompts.

Training took approximately 7–8 minutes per run, while evaluation took considerably longer because responses were generated and scored individually.

I explored batched generation to reduce evaluation time, but the batched implementation produced slightly different outputs. I retained single-prompt generation to keep the evaluation procedure consistent.

4. Discussion

The results generally supported my hypothesis. Rank 16 achieved higher win rates across all three seeds, suggesting that additional adapter capacity may help the model learn from preference data.

However, the findings were not as straightforward as I initially expected. Although rank 16 performed better than rank 8, neither configuration consistently outperformed the base model.

This suggests that increasing LoRA rank does not guarantee better preference alignment, especially with limited training data and only one epoch of fine-tuning.

The evaluation setup was another important part of the experiment. Correcting the training/evaluation overlap improved the validity of the comparison, but also meant the original baseline result could not be directly reproduced under the same metric.

Variation across seeds further showed why repeating experiments was important instead of relying on a single run.

5. Limitations and Future Work

The study relied on a single automated reward model, which may not fully reflect human preferences. Only three random seeds and 200 evaluation prompts per run were used, so the results provide an initial comparison rather than a definitive conclusion.

Changing the seed also affected dataset shuffling and selection. Different runs may therefore have used different training and evaluation examples, making it difficult to isolate training randomness.

A useful next experiment would use a fixed, independently held-out evaluation set across all runs. I would also measure trainable parameter counts, peak GPU memory usage, and end-to-end runtime to better understand the performance–efficiency trade-off.

6. Conclusion

This project compared LoRA ranks 8 and 16 for DPO fine-tuning of Qwen2.5-0.5B-Instruct. Rank 16 achieved a mean win rate of 48.83%, compared with 39.83% for rank 8, supporting the direction of my original hypothesis.

However, both configurations averaged below 50% against the base model, and the limited evaluation setup prevents broader conclusions. Beyond the rank comparison, this project highlighted the importance of reproducibility, careful evaluation design, and troubleshooting in language-model fine-tuning.